Evaluating the performance of object-oriented and data-oriented design with multi-threading in game development

David Wingqvist, Filip Wickström, and Suejb Memeti
Department of Computer Science (DIDA)
Blekinge Institute of Technology (BTH), Karlskrona, Sweden
daag19@student.bth.se, fiai18@student.bth.se, and suejb.memeti@bth.se

2022 IEEE Games, Entertainment, Media Conference (GEM) | 978-1-6654-6138-2/22/$31.00 ©2022 IEEE | DOI: 10.1109/GEM56474.2022.10017610

***Abstract*—The frame rate of a game is important for both the end-user and the developer, and for a game to be considered playable, the minimum requirement of 60 FPS should be maintained. With respect to the programming design pattern, the current industry standard is to use Object-Oriented Design (OOD). While the demand for efficient game applications is increasing, applications developed using OOD are struggling to efficiently utilize the available computing resources and consequently meet the minimum frame-rate requirements. A design pattern that may be able to cope with the current and future requirements of resource-intensive game applications is the Data-Oriented Design (DOD), which focuses on the efficient CPU memory utilization. The main difference between OOD and DOD is related to the way data is organized and accessed. While DOD is able to efficiently utilize the cache memory, programming using the data-oriented design is perceived as much more complex compared to programming applications using object-oriented design.**

In this paper, we will first develop a simple game application using both programming design patterns. Thereafter, we evaluate the performance of both implementations with respect to the overall execution time, CPU and memory utilization. Furthermore, we will develop the corresponding multi-threading versions to explore how the memory is utilized when multiple cores access data from shared cache memory. The results of the empirical evaluation show that the code sections that use DOD perform significantly faster than the corresponding parts implemented using OOD for both single-threaded and multi-threaded applications. The maximum observed speedup of 13.25 times demonstrates that games and applications developed using DOD can utilize the available resources more efficiently.

***Index Terms*—data-oriented design, object-oriented design, multi-threading, game development**

I. Introduction

The advancements of computer graphics have had a great impact on the quality of games and applications. However, while modern general purpose and graphic processing units offer high computational power, to cope with the requirements of modern games, the scale of which increases every year [1], [2], it is important to efficiently utilize the available resources.

For a game to be considered acceptable, it should maintain a throughput of 60 FPS, and anything below is perceived as unresponsive and may result with bad user experience. According to Bayliss [2], specific applications, such as virtual reality, require even higher frame rates.

The current industry standard for writing code is to use the Object-Oriented Design (OOD) pattern, mainly because it is easier to understand, write and re-use code [3]. The main focus of OOD is on objects and the code re-usability. However, the objects created in OOD are placed in memory in an unorganized way, and access to such data structures results with several fetch requests to the slower memory storage, overheads that are detrimental to the application's efficiency.

Over the past years, the gap between CPU speed and memory speed has been growing tremendously. This is further enhanced by how the OOD pattern accesses the application's data. A solution to minimize this problem is by efficiently utilizing the CPUs cache [4] [5]. A design pattern within software development that can give more control over the cache is Data-Oriented Design (DOD). This pattern focuses on data organization, i.e. it aims to utilize the CPU memory efficiently. We recall from the spatial and temporal locality principles that the location of the data in the memory is important for the cache. While OOD might not guarantee the allocation of its member variables close to each other, the DOD on the other hand is of interest both with respect to how variables are allocated and accessed, thus taking advantage of the spatial and temporal locality [6] [7].

In this paper, we will compare the performance, with respect to execution time and cache utilization, of the OOD and DOD patterns. We have developed a simple game (inspired by Slither.io online game) using both design patterns. We have used OpenMP to parallelize the game to investigate the performance benefits of the corresponding design patterns from multi-threading. Additionally, to study the scalability of the design patterns we have developed a version of the game that incurs a more complex game play. The complexity of the game is controlled by a number of parameters used to add/remove objects, enabling/disabling object movement, or increasing/decreasing the number and size of objects.

The results of this paper show that the data-oriented design patterns yields faster execution time and lower cache miss rates compared to the object-oriented design. With respect to the execution time, a maximum speedup of 13.25x was achieved, whereas with respect to cache miss rates, a performance improvement of up to 5.57x was achieved.

The rest of this paper is structured as follows: section II provides brief background information; section III summarizes, compares and contrasts this paper to the related work;

Authorized licensed use limited to: Universitaet Linz. Downloaded on August 03,2026 at 07:12:51 UTC from IEEE Xplore. Restrictions apply.

978-1-6654-6138-2/22/$31.00 ©2022 IEEE

section IV describes the design and implementation aspects of this paper, and how the experiments were designed and conducted; section V presents and discusses the results of our experiments; and section VI concludes this paper and discusses future work directions.

II. Background

This section provides background information related to memory hierarchy with special focus on CPU cache, the OOD and DOD, and the multi-threading technology.

A. Memory hierarchy and CPU cache

A computer has different layers of memory with different speeds and sizes. The memory layer that is the closest to the CPU, is the register layer which is the smallest but also the fastest memory in the computer. The second layer is the cache, which is larger but slower than registers [4]. There are different cache levels, L1, L2 and L3 [8]. L1 is the fastest and closest to the CPU but also the smallest. Every core in the multi-core CPUs has an L1 and L2 while the L3 is shared between all cores. Lower layers such as the main memory and hard drives provide larger size capacity but with slower access speed.

The cache stores data that may be needed in the future and instructions that will be executed for data manipulation. Based on how the stored data will be reused, we distinguish temporal locality (where a chunk of data is utilized again in the near future), and spatial locality (where data that is close to the recently fetched data is to be used in the near future) [6] [7].

Lengyel [5] estimates that fetching data from the RAM takes around 500+ CPU cycles, whereas access to L2 and L1 cache takes 30-50 cycles and 5-8 cycles, respectively, thus, it is recommended that memory access should be optimized.

B. Object-Oriented Design (OOD)

OOD is a design pattern within software development that focuses on objects and the re-usability of code. An object has chunks of data and functions that can be used to change the data. Specific OOD features include (1) association, which creates a relationship between two classes that allows one class to use another, and (2) inheritance, which is used to abstract common data and functionality of multiple classes.

The advantage of OOD is how fast and effective it is to write code with the help of re-using code with inheritance. By encapsulating data, and isolating changes into classes, it is easier to debug and isolate problems in the code.

The disadvantage on the other hand is that OOD is not the most efficient pattern to use in terms of performance. The creation and destruction of objects is expensive, and since objects use complex associations, it can lead to not optimal memory access performance [3].

C. Data-Oriented Design (DOD)

DOD is a design pattern that aims to organize the data to achieve efficient utilization of the CPU cache. It differs from OOD by not utilizing abstract concepts such as objects and inheritance between classes. Within DOD, data is one of the most important factors in the application, and its main focus is on how to transform the data into actions the computer can perform in an effective manner [9].

Applications written using the DOD pattern are associated with increased code complexity, maintainability effort, and reduced readability, however, the performance benefits that it offers may be considered a good trade-off.

D. Multi-threading

When an application utilizes several threads, it is called that the application is 'multithreaded'. Multithreading, as the name suggests, is the act of using several threads to divide the work between the cores of the CPU. A thread can be seen as a worker that can run separately from the main thread. Generally, the optimal method of dividing work is to allow one thread to run per available physical or virtual core.

Multicore allows the CPU to divide work between these cores, each core can only handle one thread at a time. These cores share some cache levels with each other [10]. In such context, multiple cores may have local copies of data that may also be local to another core. Updates to the local copy of one core need to be propagated to the local copies of other cores, thus keeping a consistent view of the cache, something that is known as cache coherence [11].

III. Related Work

With respect to the existing state-of-the-art articles, not much attention was given to evaluating the performance impact that the choice of the design pattern has. In what follows we will briefly summarize the related articles and discuss how our work compares and contrasts to the existing work.

Fedoseev et al. [12] conducted a case study on OOD and DOD that use a similar approach as the work performed in this paper. Two games were developed, one using DOD and the other using OOD. For development, the authors used the Unity game engine, which provides a fast and easy way to develop games, however it provides limited control to developers that may want to better optimize their games. The results of this study showed that the DOD was able to maintain a higher frame-rate and the CPU resources were utilized more efficiently. Another highlighted insight is that writing DOD code is more complex and more suitable for advanced programmers. In comparison, we explore the performance impact of multi-threading, where multiple cores share the cache, i.e. scenarios that incur higher synchronization overheads.

Mironov et al. [13] proposed an approach where data-oriented design was used to implement a solution for accelerating the backtesting trading strategies. The paper aims at comparing the OOD-based backtester trading strategy with the OOD-based one. Furthermore, the authors aim to parallelize the DOD-based approach. The results of this paper show that the DOD implementation was able to outperform the OOD implementation by about 33% and enable a more efficient utilization of multiprocessors systems' resources.

Romeo and Fiumara [14] proposed a high-level Entity Component System based (ECS) library to allow developers

Authorized licensed use limited to: Universitaet Linz. Downloaded on August 03,2026 at 07:12:51 UTC from IEEE Xplore. Restrictions apply.

quickly and easily develop applications by simply defining the logic of the program and use graph-based techniques to connect and define the data-flow of multiple subsystems. This way of representation enables the implicit declaration of program dependencies, which in some cases enables the automatic parallelization of the program components. The authors focus on different aspects including inheritance, composition, architecture, and multi-threading. To demonstrate the library, and evaluate its performance, the authors developed a particle simulation using the presented library. Comparison of the inner parallelism and no parallelism was conducted and the results show that the version with inner parallelism consistently yields to better performance, with about 65% improvement.

Bayliss [2] focus on the design process of data-oriented applications for game development. The author highlight the importance of knowledge with respect to the data (such as type, distribution, storage, and accuracy) to efficiently utilize the cache hierarchies of the target hardware. Throughout examples, Bayliss demonstrates that parsimonious designs, which may be specific for a particular problem and target architecture, may yield better overall performance. Furthermore, similarly to our study, the author discusses the benefits of the DOD when using parallel computing resources.

Nyberg [15] investigated the data-oriented pattern and different design principles were analyzed with DOD. Similarly to this paper, the main focus is on analyzing the utilization and efficiency of the CPU cache and the overall CPU time. The author, is interested to see the performance impact of various principles associated with DOD and game development, such as the removal of polymorphism at run-time, transitioning from non-contiguous to contiguous allocation, as well as decreasing the amount of data from hot-spots. The results of this paper showed that DOD has a positive impact on the overall CPU time and the reduction of cache misses.

Similarly to our work, Fontana et al. [16] investigated the performance impact of multi-threading for both design patterns in the domain of electronic design automation tools. The results showed that DOD was faster than OOD where the cache misses were fewer and the performance was faster. The work by Fontana et al. [16] is the only one of the related articles that is concerned with comparing the multi-threaded and single-threaded performance of DOD and OOD. In comparison, our paper addresses the same issue in the context of game development domain. Specifically, this paper will investigate the performance impact of the data-oriented and object-oriented design patterns when both single-threading and multi-threading is utilized.

IV. Method

To achieve the goals of this paper, a simple game was developed using the two different design patterns. Furthermore, the game was parallelized, so in total we have 4 versions of the same game, (1) single-threaded OOD (ST-ODD), (2) single-threaded DOD (ST-DOD), (3) multi-threaded OOD (MT-OOD), and (4) multi-threaded DOD (MT-DOD).

TABLE I: Systems used for the empirical evaluation.

	W1 / U1*	W2 / U2	W3/D*
CPU Type	i5-7600k	i7-8750h	i9-9900k
CPU Freq. [GHz]	4,17	4,0	4,5
# cores	4	6	8
# threads	4	12	16
L1 [KB]	256	384	512
L2 [MB]	1	1,5	2
L2 [MB]	6	9	16
RAM Size [GB]	15,9	16	16
RAM Freq. [MHz]	2133	2667	3200
OS	Windows 10±	Windows 10±	Windows 10∓

* virtual environment. The available RAM is 8GB.
± the same system configuration was used with Ubuntu 20.04.4.
∓ the same system configuration was used with Debian 11.3

The multi-threaded versions utilize OpenMP, which is a high-level parallel programming framework that provides simple compiler directives to parallelize sections of code.

This paper is concerned with analyzing the performance efficiency of the different patterns, hence each of the implemented versions of the game is executed with various parameters (which control the complexity of the game, in terms of the resource requirements) and the overall execution time and the cache miss rate is monitored.

The games were developed by the first two authors of this paper, and to mitigate the bias from our previous coding experience, we decided that each author developed a game individually using both of the patterns in an alternate order (i.e., the first author developed the game using OOD first and then DOD, whereas the other author used DOD first and then OOD), and the multi-threaded version is coded together. This would allow us to better reflect on and analyze the programming complexity effort required by both designs.

For evaluation, we used three different testing environments. The specifications of these systems is described in Section IV-A. To measure how the cache was utilized and the number of cache misses, Valgrind[1] was used, which is a profiling tool for Linux that runs separately from the application and measures different aspects depending on the input.

SFML's[2] rendering API was used to create a window and draw all the game objects. While it is simple to use and is compatible with multiple platforms, this choice did affect the final results, which is is further explained in Section V.

A. Experiments

To conduct the experiments, we used three different computing systems, and in total six environments. Table I lists the details for each of the hardware configurations. Note that each computer had two different environments. For instance W1 is the same hardware configuration as U1, the only difference is that W1 runs Windows OS whereas U1 runs Ubuntu. Note that U1 and D3 are virtualized environments, whereas the rest are host OS (in the case of W2/U2 we have dual boot).

[1]Available at: https://valgrind.org
[2]Available at: https://www.sfml-dev.org

Authorized licensed use limited to: Universitaet Linz. Downloaded on August 03,2026 at 07:12:51 UTC from IEEE Xplore. Restrictions apply.

To evaluate the performance of the considered design patterns, we developed a simple game that would stress the CPU by having as many game objects as possible in the scene. It was inspired by the online web browser game called Slither.io, where a snake eats food in order to grow larger and defeat other players. Figure 1 shows an example of how the game looks like. The main player is represented by a sphere (blue color) that collects healthy food (green), which increases the score (top left corner), whereas the poisonous food (red) should be avoided as it does the opposite. The size of the player increases as the score increases. Some of the food move in a random direction to make the game difficult. The pink circles represent enemy players, which if collided will reset the score of the main player.

The complexity of the game can be adjusted by adding/removing objects (e.g., enemy players), making objects static (e.g., poisonous food), or increasing/decreasing the size of the objects. We defined two games with different complexity, (G1) where we have only red and green circles, and only the red circles move; and (G2) where food is all static, but there are enemy players (pink circles).

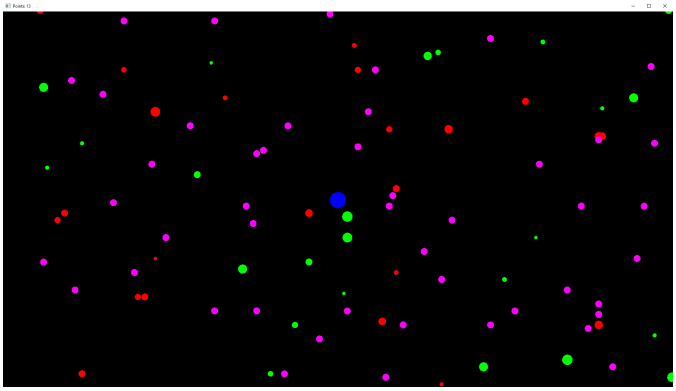


Fig. 1: An example of the game used for evaluation.

V. Results

This section presents the results with respect to the execution time, cache utilization and cache misses at different levels of cache. These metrics are used to compare the behavior of the game programmed with the two design patterns (i.e. OOD and DOD) within a sequential and parallel execution context.

Each section shows the different performance metrics for each game created for this paper. First we describe the performance of the game when executed sequentially and later when executed in parallel.

A. Sequential execution

The sequential execution of the game involves a single thread. Labels that use the *_S* postfix indicate instances where the game is executed sequentially.

1) Execution time: Figure 2 shows the execution time for the first game executed in systems W1, W2, and W3. We vary the number of objects to demonstrate the scalability of the game. The figure on the bottom shows the magnified results for the DOD version of the game executed in all systems, whereas the first one shows both OOD and DOD to easily see the performance difference between the design patterns.

Note that the execution time shows the time that was needed to produce one frame. We may observe that the DOD version of the game results with much higher performance (i.e. significantly lower execution time) compared to the OOD version regardless of the type of system it was executed on or the number of game objects. When we executed the game on system W1 with 100K we have achieved up to 9.17 times speedup. Similar behavior can be observed for game 2, the results for which are shown in Figure 3.

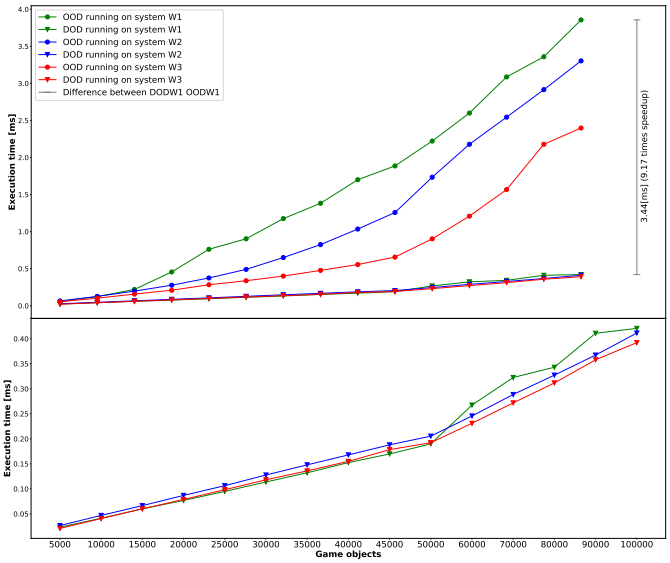


Fig. 2: Execution time (in milliseconds) for the first game executed in systems W1, W2, and W3.

2) Cache misses: Figure 4 shows the cache misses for game 1 and game 2 executed in systems U1, U2, and D. The gray colored bars show the L1 cache misses and white colored bars show LL cache misses. The dark gray colored bars correspond to the results for the OOD implementation, whereas the light-gray bars correspond to the results for the DOD implementation. Note that we have grouped the results first by the number of game objects, and then by the system it was executed on. For each system we show the cache miss rate (including L1 and LL) for both OOD and DOD.

We may observe that the cache misses for the DOD pattern is significantly lower across all systems compared to the corresponding implementation with OOD. We may observe that the cache misses for the OOD spikes after increasing the game objects to 20k for both games running on system U1. Similar behavior is seen for the other systems. The DOD on the other hand shows low number of cache misses regardless of the number of game objects or type of system.

Another observation related to cache misses is that when using OOD, last-level cache misses increases with the number of objects. A cache miss on this level incurs high overheads,

Authorized licensed use limited to: Universitaet Linz. Downloaded on August 03,2026 at 07:12:51 UTC from IEEE Xplore. Restrictions apply.

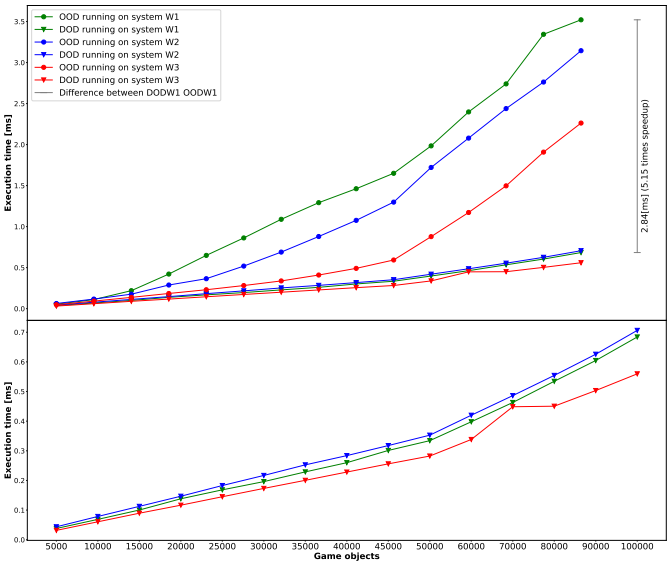


Fig. 3: Execution time (in milliseconds) for the second game executed in systems W1, W2, and W3.

because such data should be fetched from the RAM, which is much slower. For the DOD pattern on the other hand, no major increases were observed during the tests. This would most likely be because of the games being small enough to not require much cache space for DOD.

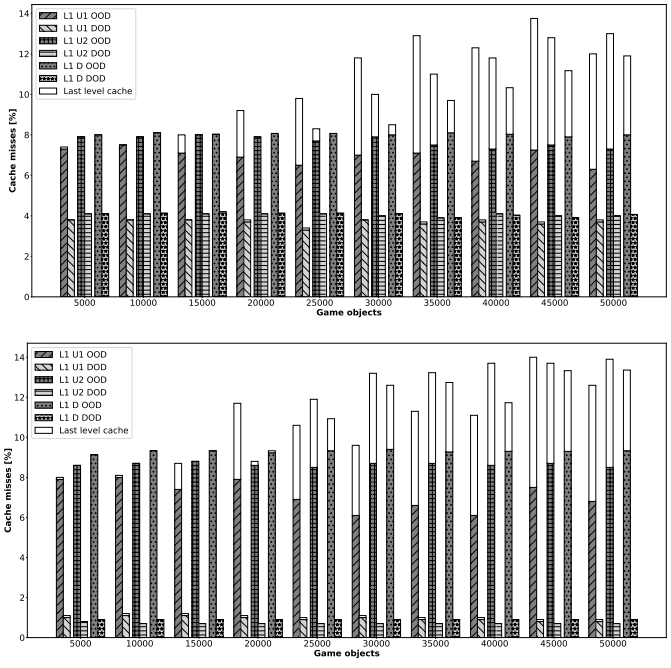


Fig. 4: Cache misses for the first (top figure) and second (bottom figure) game executed on systems U1, U2, and D.

B. Multi-threaded execution

The multi-threaded execution of the game means that multiple threads work together to solve the time and memory intensive tasks of the game. In the following sections, you may see figures where the *_NT* postfix is added to the labels in the figure, where N can be 4, 12, or 16, and corresponds to the total number of threads that a system has. For instance, *DOD W1_4T* means the game version is implemented using DOD and executed in parallel on system W1 using 4 threads.

1) Execution time: Figure 5a shows the performance comparison of the multi-threaded DOD and single-threaded DOD versions of game 1. While the y-axis shows the execution time in milliseconds, the values above the bar show the performance improvement (speedup) of DOD over OOD implementation.

Similarly to the single-threaded implementation, DOD performs significantly better for multi-threaded implementation as well. The maximum observed speedup was 13.25x when executing game 1 on system W1 with 4 threads using 90k objects. Note that in reality, a regular game often does not include over 5K game objects. Hence utilizing DOD in real-world scenarios would perhaps results with lower speedups.

While Figure 5a highlights the performance benefits from DOD relative to OOD when using multi-threading, Figure 5b highlights the performance benefits of multi-threading DOD compared to its sequential DOD version. We have observed the highest speedup of 5.44x when running the game with 100K objects on system W3 with 16 threads.

2) Cache utilization: Figure 6 shows the performance results related to cache misses of the multi-threaded implementations. Figure 6a highlights the difference with respect to cache miss rate ratio between the DOD and OOD. We may observe that DOD has significantly lower cache misses throughout all variations of game objects and systems. When we use 50k objects on system U2 with 12 threads, a maximum cache miss reduction of 5.57 times in favor of DOD was observed.

Figure 6b highlights the difference in terms of cache misses between the multi-threaded and single-threaded DOD. We may observe that there are significant improvements with respect to cache misses when using multi-threading. The maximum achieved improvement is 10.25x.

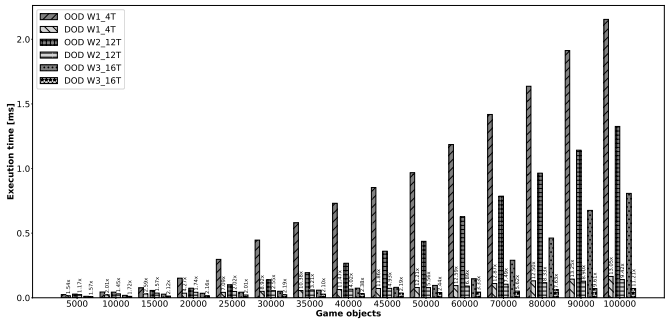
C. The programming complexity

Even though the development of the game by both authors was alternated, such that the first author developed the OOD first and then the DOD version, and the second author did it in the reverse order, both authors conclude that using OOD was much easier. Implementing the game using DOD was more complex compared to the corresponding OOD version of the game. There is a clear trade-off between the two design patterns, OOD provides simplicity in terms of code readability and re-usability at the cost of performance efficiency.

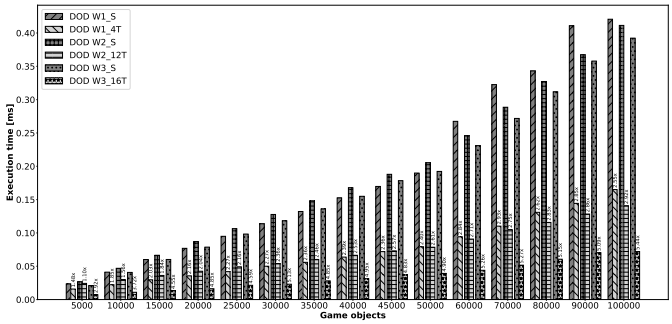
VI. Conclusion and Future Work

In this paper, we compared the performance, with respect to execution time and cache utilization, of two games developed using object-oriented and data-oriented design patterns.

Authorized licensed use limited to: Universitaet Linz. Downloaded on August 03,2026 at 07:12:51 UTC from IEEE Xplore. Restrictions apply.

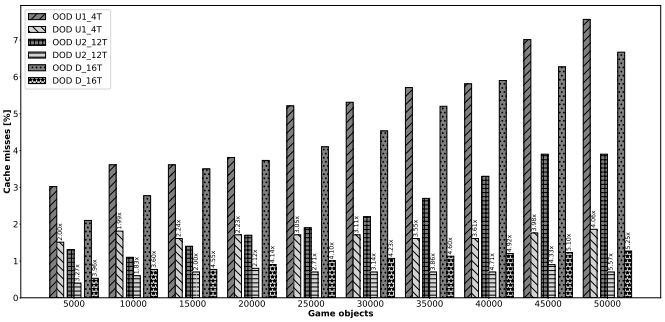


(a) The execution time of game 1 on systems W1, W2 and W3. We compare multi-threaded implementations of DOD and OOD. The values above the bars show the speedup of DOD relative to the OOD.

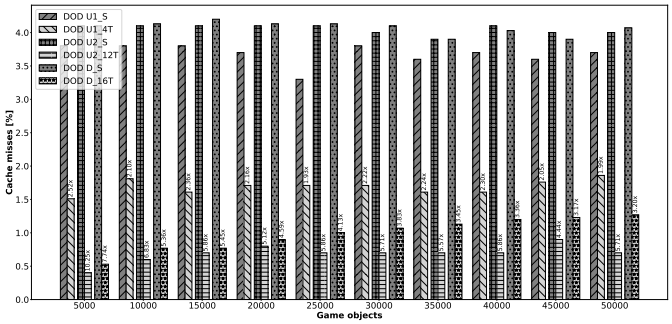


(b) The execution time of the single-threaded and multi-threaded DOD version of game 1 executed on systems W1, W2 and W3. The values above the bars show the multi-threading speedups.

Fig. 5: Performance comparison of (a) MT-DOD with MT-OOD, and (b) MT-DOD with ST-DOD of game 1.



(a) The cache miss rate of game 1 on systems U1, U2 and D. We compare the DOD and OOD implementations. The labels show the improvement rate of cache misses of DOD relative to OOD.



(b) The cache miss rate of the single-threaded and multi-threaded DOD versions of game 1 executed on systems U1, U2 and D. The labels show the improvement from multi-threading.

Fig. 6: Comparing cache utilization of (1) MT-DOD with MT-OOD, and (2) MT-DOD with ST-DOD of game 1.

Furthermore, we investigated the performance benefits of the corresponding design patterns when using multi-threading.

We have executed experiments using various systems comprising CPUs with different number of cores/threads and cache sizes, and we have varied the number of objects present on the game to observe the scalability of the design patterns.

We have observed that the game developed using DOD performs significantly better than the OOD both with respect to execution time and the cache miss rates. Regarding the execution time, a maximum speedup of 13.25x was achieved when using DOD compared to the OOD. Whereas, with respect to cache miss rates, a performance improvement of up to 5.57x was achieved when using multi-threading and DOD.

Future work may focus on investigating the performance benefits of DOD with real-world scale games. Furthermore, the difference with respect to programming complexity between the two design patterns should be investigated.

REFERENCES

[1] A. Kuo, J. L. Hiler, and R. J. Lutz, "From super mario to skyrim: A framework for the evolution of video game consumption," *Journal of Consumer Behaviour*, vol. 16, no. 2, pp. 101–120, 2017.

[2] J. D. Bayliss, "The data-oriented design process for game development," *Computer*, vol. 55, no. 5, pp. 31–38, 2022.

[3] B. Dathan and S. Ramnath, *Object-Oriented Analysis, Design and Implementation*. Springer, 2015, ch. 1,3, ISBN: 9783319242781.

[4] C. Carvalho, "The gap between processor and memory speeds," in *Proc. of IEEE International Conference on Control and Automation*, 2002.

[5] E. Lengyel, *Game engine gems 2*. CRC Press, 2011, ch. 15, pp. 251–261, ISBN: 9781568814377.

[6] B. Jacob, D. Wang, and S. Ng, *Memory systems: cache, DRAM, disk*. Morgan Kaufmann, 2010, ch. 1.2, pp. 62–63, ISBN: 9780123797513.

[7] J. Handy, *The cache memory book*. Morgan Kaufmann, 1998, ch. 1, pp. 5–8, ISBN: 9780123229809.

[8] L. Null and J. Lobur, *Essentials of Computer Organization and Architecture*, 5th ed. Jones & Bartlett Publishers, 2018, ch. 6.4, ISBN: 9781284123036.

[9] R. Fabian, "Data-oriented design," 2013. [Online]. Available: https://www.dataorienteddesign.com/dodmain.pdf

[10] N. Shavit and M. Herlihy, *The art of multiprocessor programming*. Morgan Kaufmann, 2012, ch. 1, ISBN: 9780123973375.

[11] D. Culler, J. P. Singh, and A. Gupta, *Parallel computer architecture: a hardware/software approach*. Gulf Professional Publishing, 1999.

[12] K. Fedoseev, N. Askarbekuly, A. E. Uzbekova, and M. Mazzara, "Application of data-oriented design in game development," *Journal of Physics: Conference Series*, vol. 1694, no. 1, p. 012035, dec 2020.

[13] T. Mironov, L. Motaylenko, D. Andreev, I. Antonov, and M. Aristov, "Comparison of object-oriented programming and data-oriented design for implementing trading strategies backtester," in *Proc. of the International Scientific and Practical Conference*, vol. 2, 2021, pp. 124–130.

[14] V. Romeo and G. Fiumara, "Analysis of entity encoding techniques, design and implementation of a multithreaded compile-time entity-component-system c++ 14 library," Ph.D. dissertation, University of Messina, 2016.

[15] F. Nyberg, "Investigating the effect of implementing data-oriented design principles on performance and cache utilization," p. 59, 2021.

[16] T. A. Fontana, S. Almeida, R. Netto, V. Livramento, C. Guth, L. Pilla, and J. L. Güntzel, "Exploiting cache locality to speedup register clustering," in *Proceedings of the 30th Symposium on Integrated Circuits and Systems Design: Chip on the Sands*, 2017, pp. 191–197.

Authorized licensed use limited to: Universitaet Linz. Downloaded on August 03,2026 at 07:12:51 UTC from IEEE Xplore. Restrictions apply.